



Day 2 — Jenkins Pipeline & Jenkinsfile Development

Building production-grade pipelines with stages, parameters, credentials, and failure handling.

Learning Objectives

Pipeline Basics

Build Declarative Pipelines, write Jenkinsfiles, organize stages & steps

Configuration

Set environment variables, accept runtime parameters, configure triggers

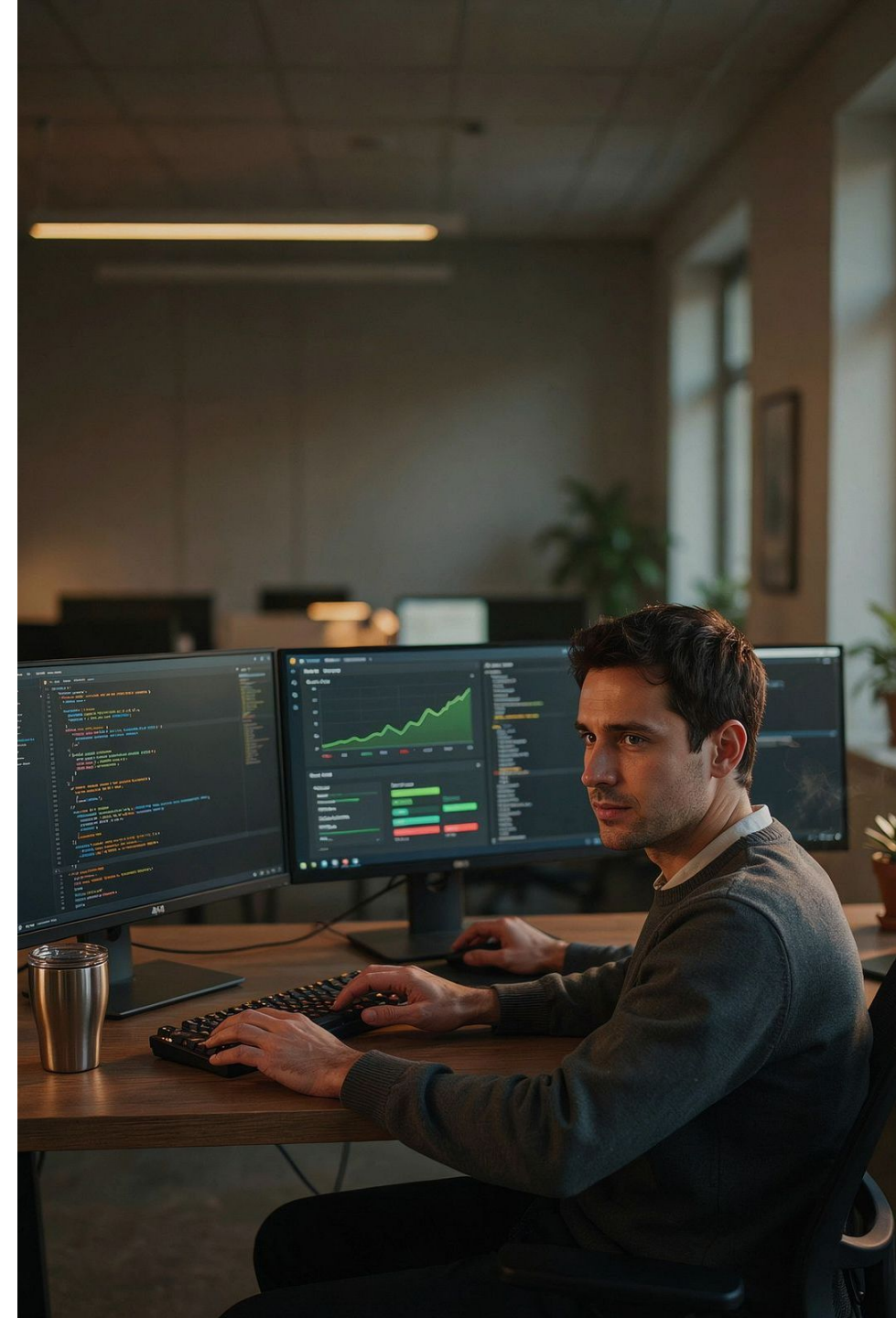
Resilience

Handle credentials securely, configure timeout & retry, define post-build actions

Troubleshooting

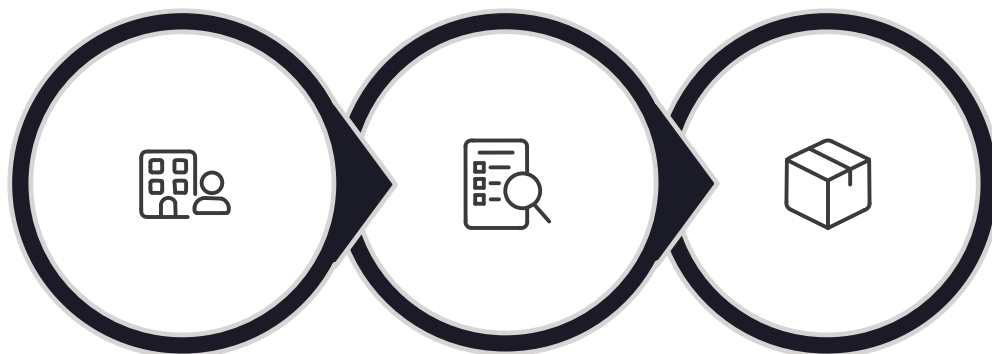
Diagnose and fix basic Pipeline failures

📄 Hands-on: Build a multi-stage Jenkinsfile.



From Basic Pipeline to Real Pipeline

Day 1 — What We Built



Build

Test

Package

Real-World Pipelines Need More

A production Pipeline needs configuration, control, failure handling, and repeatability.

Parameters

Runtime inputs

Env Variables

Configuration values

Retry & Timeout

Failure control

Post Actions

Reports & notifications

Declarative Pipeline Structure

```
pipeline {  
  agent any  
  stages {  
    stage('Build') {  
      steps {  
        echo 'Building application'  
      }  
    }  
    stage('Test') {  
      steps {  
        echo 'Running tests'  
      }  
    }  
  }  
}
```

Core Structure

pipeline

Wraps the entire definition

agent

Where the Pipeline executes

environment

Shared config values

stages / stage

Logical delivery phases

steps

Actual commands executed

post

Actions after execution

Pipeline, Agent, Stages & Steps

1

pipeline

Defines the complete Declarative Pipeline — the outermost block.

2

agent

Defines where the Pipeline executes. `agent any` uses any available executor.

3

stages

Contains the major phases: Checkout → Build → Test → Package.

4

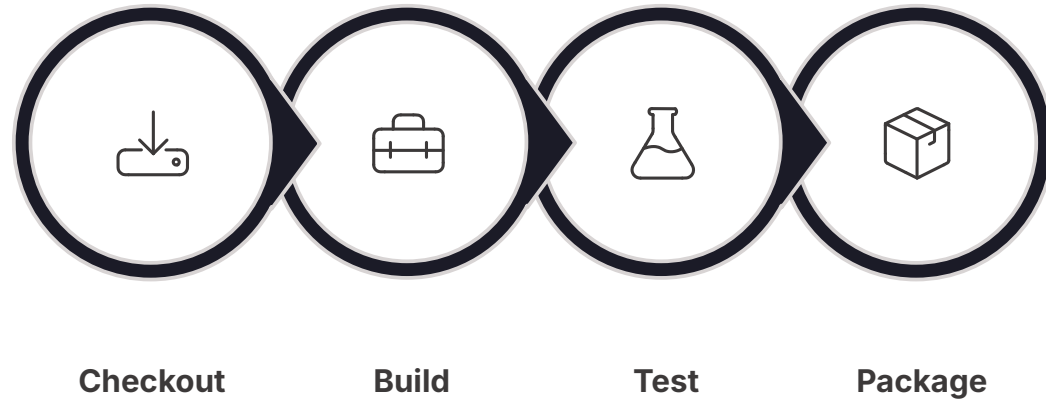
steps

Contains the actual commands executed within each stage.

Building a Multi-Stage Pipeline

```
pipeline {  
  agent any  
  stages {  
    stage('Checkout') {  
      steps { echo 'Getting source code' }  
    }  
    stage('Build') {  
      steps { echo 'Building application' }  
    }  
    stage('Test') {  
      steps { echo 'Running tests' }  
    }  
    stage('Package') {  
      steps { echo 'Creating artifact' }  
    }  
  }  
}
```

Pipeline Visualization



Each stage represents a meaningful delivery activity.

SECTION 2

What is a Jenkinsfile?

A **Jenkinsfile** is a text file containing the Pipeline definition — stored in the same source-control repository as the application.

Version Controlled

Changes tracked alongside code

Reviewable

Part of the pull request process

Traceable

Full audit history

Repeatable

Consistent builds every time



Pipeline from Jenkinsfile



Development Flow

- 1 Modify Code / Jenkinsfile**
- 2 Commit to Git Repository**
- 3 Jenkins Executes Pipeline**

Pipeline configuration becomes part of the development lifecycle.

Environment Variables

```
pipeline {  
  agent any  
  environment {  
    APP_NAME = 'customer-api'  
    APP_ENV = 'dev'  
  }  
  stages {  
    stage('Build') {  
      steps {  
        echo "Building ${APP_NAME}"  
        echo "Environment: ${APP_ENV}"  
      }  
    }  
  }  
}
```

Common Uses

- Application & environment name
- Build configuration & URLs
- Tool configuration & paths

Jenkins Built-in Variables

- `env.BUILD_NUMBER` — unique build ID
- `env.JOB_NAME` — pipeline name
- `env.WORKSPACE` — working directory

❏ Avoid hardcoding environment-specific values throughout Pipeline steps.

Parameterized Pipelines

```
parameters {  
  choice(  
    name: 'ENVIRONMENT',  
    choices: ['dev', 'qa', 'uat'],  
    description: 'Deployment environment'  
  )  
  booleanParam(  
    name: 'RUN_TESTS',  
    defaultValue: true  
  )  
}  
steps {  
  echo "Env: ${params.ENVIRONMENT}"  
  echo "Tests: ${params.RUN_TESTS}"  
}
```

Why Parameters?

The same Pipeline supports different runtime requirements without modifying Pipeline code.

choice

Select from a list (dev / qa / uat)

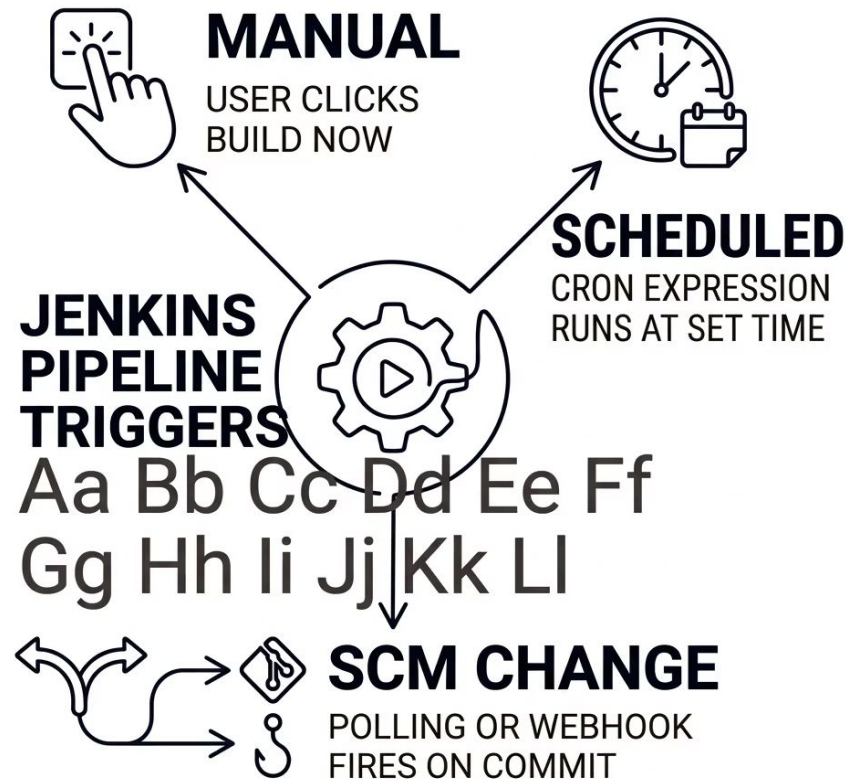
booleanParam

Toggle a flag (true / false)

string

Free-text input value

Build Triggers



Common Trigger Approaches

→ Manual

User initiates the build

→ Scheduled

```
triggers { cron('H 2 * * *') }
```

→ SCM Polling / Webhook

Fires on repository change



Git & webhook integration is covered in depth on Day 3.

SECTION 6

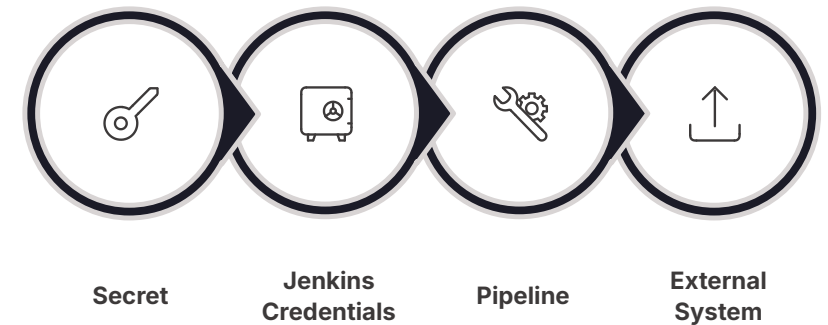
Credentials in Pipelines

❌ Never Do This

```
environment {  
  PASSWORD =  
  'MyPassword123'  
}
```

Secrets stored directly in Pipeline code are exposed in version control.

✅ Better Approach



Use Jenkins-managed credentials. Detailed credential types covered on Day 4.

Timeout & Retry

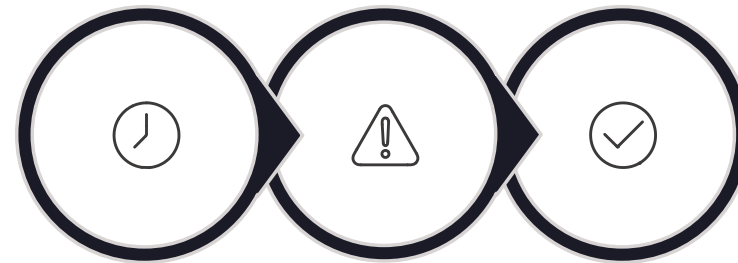
Timeout

```
stage('Test') {  
  steps {  
    timeout(time: 10, unit: 'MINUTES') {  
      echo 'Running tests...'  
    }  
  }  
}
```

Protects Jenkins from hanging processes, unresponsive services, and wasted executors.

Retry

```
stage('Publish') {  
  steps {  
    retry(3) {  
      echo 'Publishing artifact...'  
    }  
  }  
}
```



Attempt 1
Failed

Attempt 2
Failed

Attempt 3
Success

Useful for temporary failures — not for hiding genuine application problems.

Pipeline Post Actions

```
post {  
  success {  
    echo 'Pipeline completed successfully'  
  }  
  failure {  
    echo 'Pipeline failed'  
  }  
  always {  
    echo 'Pipeline execution completed'  
  }  
}
```

Common Conditions

always

Runs regardless of result

success

Only on success

failure

Only on failure

unstable

Partial failures

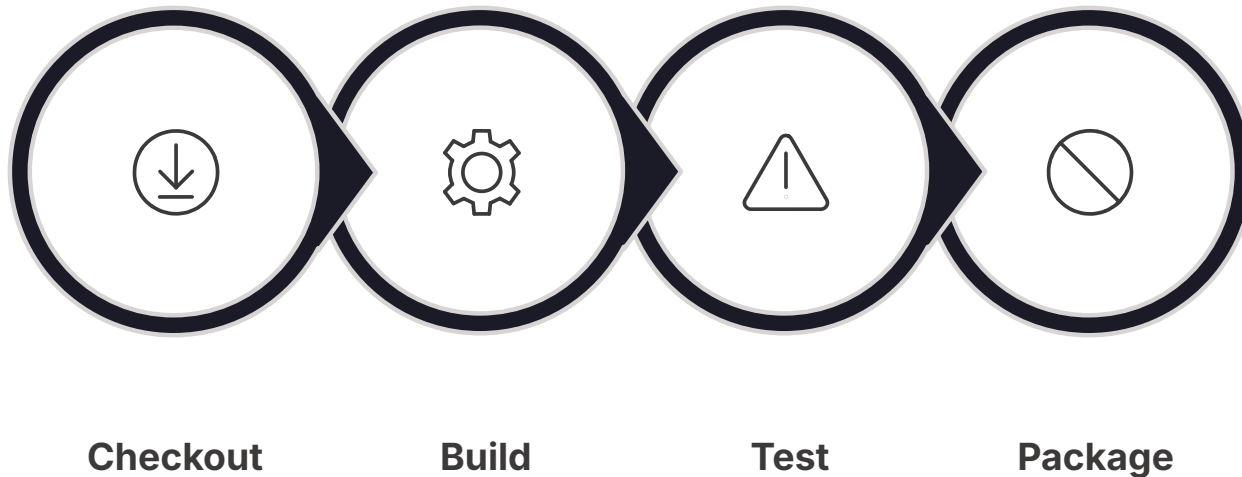
changed

Result differs from last run

Typical Uses

Notifications, cleanup, reports, logging, artifact handling.

Understanding Pipeline Failures



If a required stage fails, subsequent stages normally do not continue.

Troubleshooting Approach

- 1 Identify the Failed Stage
- 2 Open Console Output
- 3 Find the Error
- 4 Fix & Re-run

□ Console Output is one of the first places to investigate Pipeline failures.

Putting Everything Together

```
pipeline {
  agent any
  parameters {
    choice(name: 'ENVIRONMENT',
      choices: ['dev', 'qa'])
  }
  environment { APP_NAME = 'sample-app' }
  stages {
    stage('Build') {
      steps {
        echo "Building ${APP_NAME}"
        echo "Build #${env.BUILD_NUMBER}"
      }
    }
    stage('Test') {
      steps {
        timeout(time: 5, unit: 'MINUTES') {
          echo 'Running tests'
        }
      }
    }
    stage('Package') {
      steps {
        retry(2) { echo 'Creating package' }
      }
    }
  }
  post {
    success { echo 'Pipeline successful' }
    failure { echo 'Pipeline failed' }
    always { echo 'Pipeline completed' }
  }
}
```

Concepts Used

 Parameters	 Environment
 Stages & Steps	 Timeout
 Retry	 Post Actions

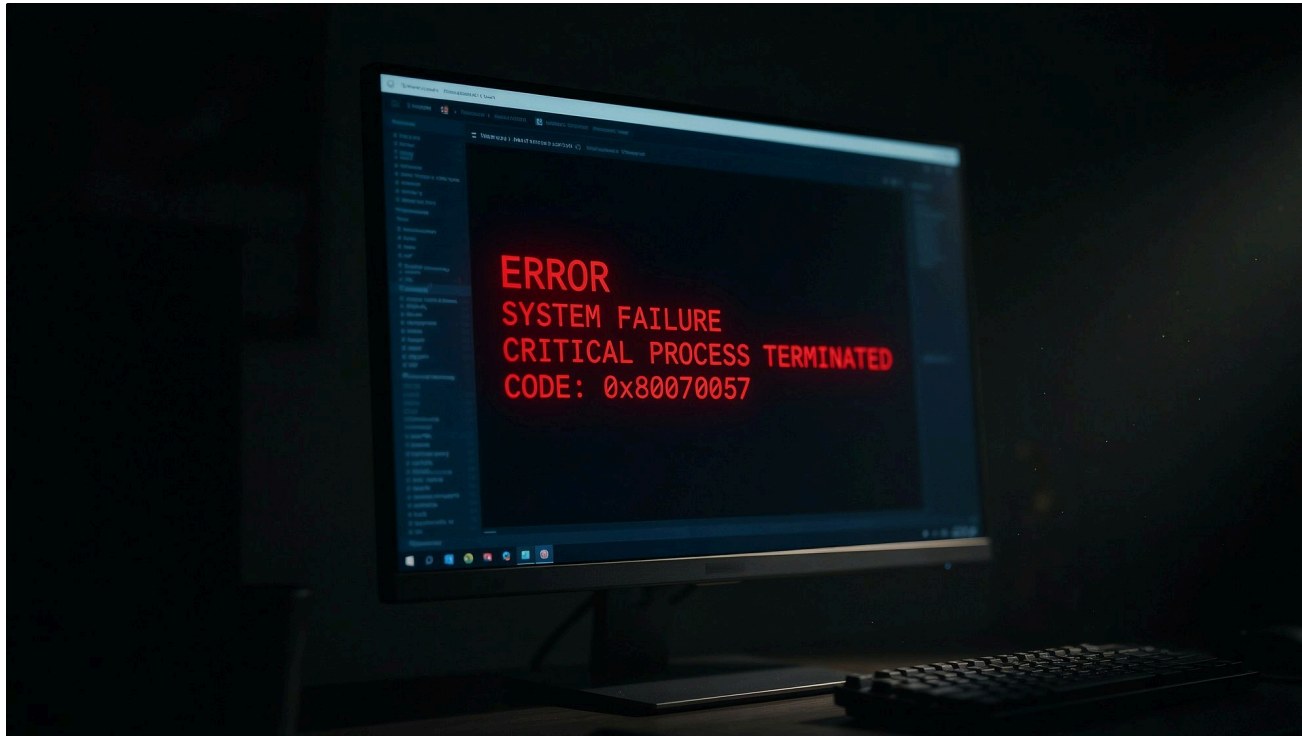
Day 2 Hands-On Lab

Build a Multi-Stage Jenkinsfile

Convert the basic Day 1 Pipeline into a configurable, resilient Jenkinsfile.

01	02	03
Create Repo Structure	Build → Test → Package	Add Env Variables
jenkins-pipeline-lab/Jenkinsfile	Implement the three core stages	APP_NAME, APP_ENV
04	05	06
Add Parameter	Add Timeout & Retry	Add Post Actions
Choice: dev / qa / uat	5-min timeout on Test; retry(3) on operations	success / failure / always
07		
Execute & Review		
Check stages, build number, parameters, Console Output		

Lab Challenge



Intentionally Fail a Stage

Introduce a deliberate failure in one of your Pipeline stages, then use **Console Output** to identify exactly where and why it failed.

Review Checklist

- Which stage failed?
- What does the error message say?
- Did post actions still execute?
- What is the final Pipeline status?

📄 Console Output is your first debugging tool — learn to read it well.

Day 2 — Key Takeaways

Declarative Structure

pipeline → agent → stages → steps → post

Jenkinsfile

Store Pipeline definition in source control alongside your app

Env & Parameters

Avoid hardcoding — use variables and runtime inputs

Resilience

Timeout prevents hangs; retry handles transient failures

Credentials

Never hardcode secrets — use Jenkins Credentials Store

Troubleshooting

Start with Console Output to diagnose failures